

Team ID: DSCPP-III-2026-T186

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T186
Semester: 3rd
Section: A / D / H
Domain: Graph Algorithms (C++)

Team Members

1. Tejas Jain – 2028228 (SEC-A)
2. Yash Nigam – 2028289 (SEC-H)
3. Apoorv Negi – 2027640 (SEC-D)

Mentor

Mr. Navneet Rajpoot

Interactions completed: _____

Approved Project Name: EcoGraph – Graph Based Algorithmic Waste Management.

Problem Statement & Motivation

Problem Statement

Most municipal waste routes are **fixed in advance** and never actually reflect the shortest path between homes, bins, and the depot – so trucks burn extra fuel, take longer, and tie up more staff than they need to. Hazardous waste rides along on the same route as ordinary trash, even though it really needs immediate, zone-restricted pickup, and because schedules run on a fixed calendar, some bins get emptied too often while others overflow between visits.

Motivation

- ▶ Fixed routes don't account for the city's actual road network, so trucks aren't taking the shortest path
- ▶ Hazardous waste needs to be disposed of immediately, not left to wait its turn, and routed clear of residential areas rather than trucked straight through them
- ▶ Calendar-based pickups send labor to bins that aren't even close to full

Target Users & Stakeholders

Who This Is Built For

- ▶ Municipal waste departments and truck operators
- ▶ Residents and commercial establishments on the route
- ▶ Public health – getting hazardous waste cleared faster and more safely

Objectives

Project Objectives

1. Represent the city as a weighted graph, with the depot, homes, junctions, and bins as nodes, each one tagged by zone type – residential, commercial, or industrial.
2. Work out collection routes with Dijkstra's algorithm, letting the truck switch between two modes as needed: Priority or FIFO.
3. Flag hazardous waste the moment it's detected and send it straight out on a zone-constrained path that steers clear of residential and commercial areas.
4. Base collection frequency on how full a bin actually is, not on a fixed calendar.
5. Put all of this behind a REST API, with the route computation playing out live and animated on the web frontend.

Proposed Solution – Approach

Proposed Approach

1. **City layout** – at startup, the server reads the depot, homes, junctions, bins, and their zone tags straight from a JSON config file.
2. **Routing engine** – shortest paths between any two nodes come from Dijkstra's algorithm, run in either a plain or a zone-constrained variant.
3. **Route builders** – multi-stop collection runs get sequenced by whichever strategy is active, Priority or FIFO.
4. **Scheduler** – a location's fill-rate and how long it's been since its last pickup decide whether it's due for a visit.
5. **Delivery** – everything comes back as JSON over a REST API and plays out live in a React frontend built with Cytoscape.js.

Proposed Solution – Key Features

Key Features

- ▶ A city model built as a graph, with every node tagged by zone
- ▶ Shortest-path routing that toggles between Priority and FIFO
- ▶ Waste gets classified through rule-based keyword matching
- ▶ Hazardous waste skips the normal queue entirely and routes out on a zone-constrained path right away
- ▶ Collection frequency adjusts on its own based on how full each location is
- ▶ Routes animate live, with hazardous ones visually called out

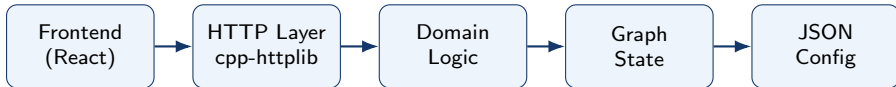
Each of these maps to one of the five REST endpoints and a matching domain-layer C++ class – see slides 10–14.

Integration of PBL Course Concepts

Course	Concepts Applied
Data Structures in C	Graph (adjacency list), queue (FIFO), priority queue / min-heap, hash map (<code>unordered_map</code>), and path reconstruction via vector/stack
OOPs with C++	Encapsulation – Graph keeps its node/adjacency maps private and only reachable through public methods; Abstraction – Scheduler and Classifier hide their heap and keyword logic behind simple calls like <code>getDueLocations()</code> and <code>classifyWaste()</code> ; Inheritance – <code>PriorityRouteBuilder</code> and <code>FIFORouteBuilder</code> both derive from a shared <code>RouteStrategy</code> base; Polymorphism – each one overrides a virtual <code>buildRoute()</code> to behave differently at runtime

Integration: The city graph – built on adjacency lists, hashing, heaps, and queues from DSA – comes together as a set of C++ classes that put all four OOP pillars to use, tying Dijkstra, routing, and scheduling into one working backend.

System Architecture Overview



- ▶ The graph lives in memory inside one persistent process, so there's no re-parsing on every request
- ▶ The backend is layered – HTTP layer → controllers → pure, testable domain classes
- ▶ Static files and the API share the same origin, so CORS never becomes an issue

Architecture – Components & Data Flow

Major Components

- ▶ Graph – Node, Edge, zone tags, held in an `unordered_map` adjacency list
- ▶ Routing engine, covering plain and constrained Dijkstra plus the Priority/FIFO route builders
- ▶ Classifier, which sorts waste using keyword rules
- ▶ Scheduler, which works out due dates from fill-rate
- ▶ REST controllers, one per endpoint

Data / Control Flow

- ▶ The frontend calls `/route`, `/classify`, `/schedule`, and `/route/hazard`
- ▶ The backend loads `city.json` just once, at startup
- ▶ Fill-rate and last-collected values change in memory as locations get visited
- ▶ Every response comes back as a JSON path plus cost, then animates on the graph canvas

Domain Model – Core Data Structures

Node

- ▶ id and type (Depot/Home/Junction/Bin)
- ▶ zone (Residential/Commercial/Industrial/Arterial)
- ▶ fillRate, capacity, lastCollected

Edge

- ▶ to and weight (travel cost)
- ▶ Its own zone tag, separate from either endpoint's

Graph class

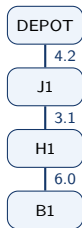
- ▶ Backed by `unordered_map<string, vector<Edge>>`
- ▶ Node lookup runs $O(1)$ on average, edge iteration $O(\text{degree})$
- ▶ Exposes `loadFromJson()`, `neighbors()`, `toJson()`

```
{"id":"H1","type":"home","zone":"residential","fillRate":5.0,"capacity":50}
```

Structure / Algorithm	Short Definition	Used For
Graph (Adjacency List)	Each node holds a list of its outgoing weighted edges	City map: depot, homes, junctions, bins
Hash Map (unordered_map)	Key $\rightarrow$ value store, $O(1)$ average lookup	Node ID $\rightarrow$ adjacency list / node data
Queue (FIFO)	First element enqueued is the first dequeued	Sequences homes in FIFO collection mode
Priority Queue (Min-Heap)	Smallest-key element always dequeued next	Dijkstra's frontier – nearest node first
Priority Queue (Max-Heap)	Largest-key element always dequeued next	Scheduler – ranks locations by overdue-ness
Dijkstra's Algorithm	Greedy shortest path via min-heap relaxation	Depot / home / bin routing
Constrained Dijkstra	Dijkstra skipping forbidden-zone edges	Hazardous waste routing
Vector (Path Reconstruction)	Backtrack predecessor pointers into an ordered list	Rebuilds the final route

Graph Storage – Adjacency List Architecture

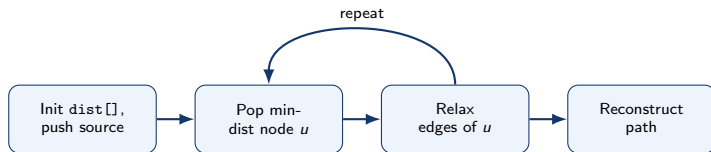
The graph sits on an `unordered_map<string, vector<Edge>>`, which keeps node lookup at $O(1)$ and edge iteration at $O(\text{degree})$.



Node	Adjacency List
DEPOT	(J1, 4.2, arterial)
J1	(DEPOT,4.2,art.), (H1,3.1,resid.)
H1	(J1,3.1,resid.), (B1,6.0,indust.)
B1	(H1, 6.0, industrial)

- ▶ Every Edge carries its own zone tag, which is what lets the constrained variant treat it as impassable
- ▶ Everything loads once from `city.json` at startup, and edges default to bidirectional

Dijkstra's Algorithm – How It Works

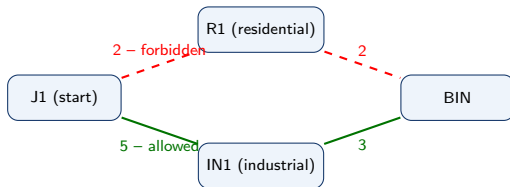


1. Set $\text{dist}[\text{source}] = 0$ and everything else to ∞ , then push the source into a min-heap keyed by distance.
2. Pop whichever node u currently has the smallest tentative distance.
3. **Relax** edge (u, v, w) : if $\text{dist}[u] + w$ beats $\text{dist}[v]$, update $\text{dist}[v]$ and push v .
4. Walk the predecessor map back to the source to reconstruct the path.

Complexity: $O((V + E) \log V)$ with a binary heap.

Custom Constrained Dijkstra for Hazardous Routing

EcoGraph adds a single extra parameter, `forbiddenZones`. During relaxation, any edge whose zone falls in that set gets skipped entirely – as far as the algorithm's concerned, it isn't even there.

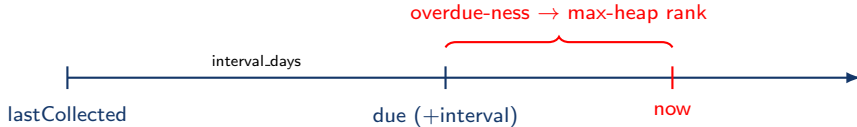


- ▶ The shorter path through R1 costs only 4, but it's residential, so it's **skipped** – the algorithm takes J1→IN1→BIN instead, cost 8, staying entirely in industrial zones
- ▶ Every hazardous dispatch calls this with `forbiddenZones = {Residential, Commercial}`; if no path clears those zones, it returns HTTP 422

Fill-Rate & Frequency Management – Interval Formula

Each home or bin stores `fillRate` (units/day), `capacity`, and `lastCollected`. From these, the scheduler works out a visit interval per location:

```
interval_days = capacity / fillRate, clamped to [1,14] days
```



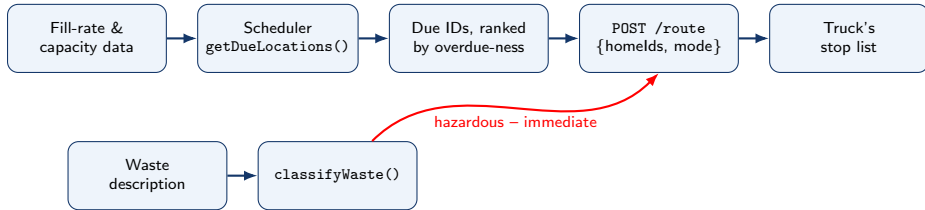
- ▶ A **floor of 1 day** keeps a fast-filling bin from demanding more than one visit per day
- ▶ A **ceiling of 14 days** means no location ever waits more than two weeks, no matter how slowly it fills
- ▶ A location becomes due once $\text{now} \geq \text{lastCollected} + \text{interval_days}$; visiting it calls `markCollected()`, which resets `lastCollected` to `now`

Fill-Rate & Frequency Management – Worked Example

Location	Capacity	fillRate	interval_days	Last Collected	Status Today
H1	50	5/day	$50/5 = 10$	9 days ago	Not due (1 day left)
H2	100	20/day	$100/20 = 5$	6 days ago	Due – 1 day overdue
B1	300	40/day	$300/40 = 7.5$	10 days ago	Due – 2.5 days overdue
B2	100	2/day	$100/2 = 50$ → 14	20 days ago	Due – 6 days overdue

The max-heap serves the most overdue first: **B2 (6d)** → **B1 (2.5d)** → **H2 (1d)** – H1 stays off today's list.

Selecting Truck Stops – Frequency + Waste Type



- ▶ Every run, the scheduler's due list feeds straight into the `homeIds` array sent to `POST /route` – that's the actual mechanism deciding which stops the truck makes
- ▶ Anything the classifier flags as **hazardous** skips the due list altogether and goes out immediately through `POST /route/hazard`
- ▶ Non-hazardous waste never jumps the schedule – it just waits for its fill-rate interval to run out, as covered on the previous slide

REST API – How It Works & Design

The frontend and backend communicate over plain HTTP: the browser sends a JSON request to a resource path, the C++ server responds with a JSON body – no session state is kept on either side between calls.

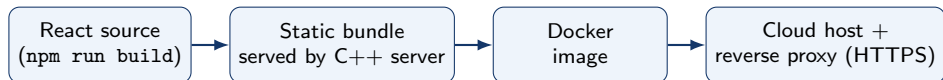
Endpoint	Purpose
GET /graph	Full node/edge list as JSON, including zone tags
POST /route	Home list + mode (priority/fifo) → path & cost
POST /classify	Waste description → bin type & hazard flag
POST /route/hazard	Hazardous location → zone-constrained route
GET /schedule	Homes/bins due for collection today

Uniform error format {ok:false, error, message} with HTTP 400/404/422/500 as appropriate.

Frontend Visualization – React + Cytoscape.js

- ▶ **React** – component-based UI (controls, mode toggle, waste-input form, schedule sidebar) with state managed via hooks; built with Vite into a static bundle.
- ▶ **Cytoscape.js** – wrapped inside a dedicated React component for the graph canvas; a graph-drawing library purpose-built for nodes-and-edges visualization.
- ▶ **Render pipeline:** GET /graph → build Cytoscape elements → style by node type/zone → auto-layout, all inside a useEffect.
- ▶ **Route animation:** walk each returned path segment, sequentially highlighting edges and pulsing nodes; hazardous routes render in a distinct dashed/red style.
- ▶ **Live schedule:** periodic GET /schedule polling flags due locations on the map without a page reload.

Deployment – Hosting the Live Demo



- ▶ `npm run build` produces a static bundle; the C++ server's static mount serves it directly – single-origin preserved, no CORS
- ▶ Backend binary + built frontend packaged into a Docker image for a reproducible deploy
- ▶ Hosted on a small cloud VM / free-tier PaaS (Render, Railway, Fly.io), reverse-proxied through Nginx/Caddy for HTTPS
- ▶ `city.json` / schedule state persisted via a mounted volume so restarts don't lose data

Technology Stack

Backend

- ▶ Programming Language: C++
- ▶ HTTP Framework: `cpp-httplib` (single-header)
- ▶ JSON Library: `nlohmann/json` (header-only)
- ▶ Build System: CMake

Frontend & Tooling

- ▶ React (Vite) for the UI, built to a static bundle
- ▶ Cytoscape.js for graph rendering and route animation
- ▶ Version Control: Git / GitHub

Development Methodology

1. **Foundations** – graph data model, JSON loader, plain Dijkstra, unit tests
2. **Core Logic** – constrained Dijkstra, classifier, Priority/FIFO routers, scheduler
3. **HTTP API** – all 5 REST endpoints wired and verified against sample data
4. **Frontend Integration** – React UI, Cytoscape.js rendering, animation, hazard styling, schedule view
5. **Deployment** – containerize backend + built frontend, host for a live demo

Each phase has an explicit exit criterion (e.g. “test binary prints correct shortest paths”) before the team moves to the next layer.

Initial Progress Achieved

- ▶ Background / literature study
- ▶ Problem statement and requirements identified
- ▶ System architecture prepared
- ▶ Technology stack finalized
- ▶ Implementation plan and team roles outlined

Individual Contribution

Member	Role	Contribution
Tejas Jain	Team Lead	Problem analysis, system architecture, REST API & domain model design, modified (constrained) Dijkstra algorithm, frequency-handling logic, waste classification, technology stack selection, and PPT preparation
Apoorv Negi	Member	Analysis and research on the modified (constrained) Dijkstra algorithm, technology stack selection, and report preparation
Yash Nigam	Member	Analysis and research on frequency-handling logic, waste classification, technology stack selection, and report preparation

Roadmap & Expected Outcomes

Roadmap

1. Phase-I: Proposal, Architecture & API Design
2. Phase-II: Graph Engine, Algorithms, REST API, Frontend
3. Phase-III: Full Live Demo, Deployment

Expected Outcomes

- ▶ Working C++ backend with an animated web frontend
- ▶ Verified shortest-path and constrained-route correctness
- ▶ Demonstrated fill-rate-driven dynamic scheduling
- ▶ Deployed and reachable as a live, hosted demo

References

1. Cormen, Leiserson, Rivest, Stein – *Introduction to Algorithms* (Dijkstra's Algorithm)
2. `cpp-http-lib` – github.com/yhirose/cpp-http-lib
3. `nlohmann/json` – github.com/nlohmann/json
4. React Documentation – react.dev
5. Cytoscape.js Documentation – js.cytoscape.org

Thank You
Questions & Suggestions